Name : Bhoomi Mangesh Naik
Class : D15C
Roll No. : 34
Practical No. : 5

Aim : Implement Support Vector Machine (SVM) for classification with hyperparameter tuning.

Theory :

1. **Dataset Source**

Source: Kaggle – Heart Disease UCI Dataset
Link: https://www.kaggle.com/datasets/ronitf/heart-disease-uci

2. **Dataset Description (Detailed)**

Dataset Name: Heart Disease UCI
Goal: Predict whether a person has heart disease or not.
Size:

- **Rows:** 303
- **Columns:** 14

Target Variable

- target
  - 1 → Heart disease present
  - 0 → Heart disease absent

Features (Inputs)

Feature	Meaning
age	Age of patient
sex	1 = male, 0 = female
cp	Chest pain type (0–3)
trestbps	Resting blood pressure
chol	Serum cholesterol
fbs	Fasting blood sugar
restecg	Resting ECG result
thalach	Max heart rate achieved

exang	Exercise induced angina
oldpeak	ST depression
slope	slope of peak exercise ST
ca	number of major vessels (0–3)
thal	thalassemia category

Dataset Characteristics

Binary classification
Mix of numerical + categorical features
Works well with SVM after scaling

3. **Mathematical Formulation of SVM**

SVM tries to find the **best separating hyperplane**:

Hyperplane:

$$w^T x + b = 0$$

Objective:

Maximize margin (distance between classes)

$$\min \frac{1}{2}||w||^2$$

With constraints:

$$y_i(w^T x_i + b) \geq 1$$

Soft Margin SVM:

$$\min \frac{1}{2}||w||^2 + C\sum_{i=1}^{n} \xi_i$$

Where:

- CC controls penalty of misclassification
- ξiξi are slack variables

Kernel Trick

Instead of linear separation, map to higher dimension:

$$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

Common kernels:

- Linear
- Polynomial
- RBF
- Sigmoid

4. **Algorithm Limitations (SVM)**

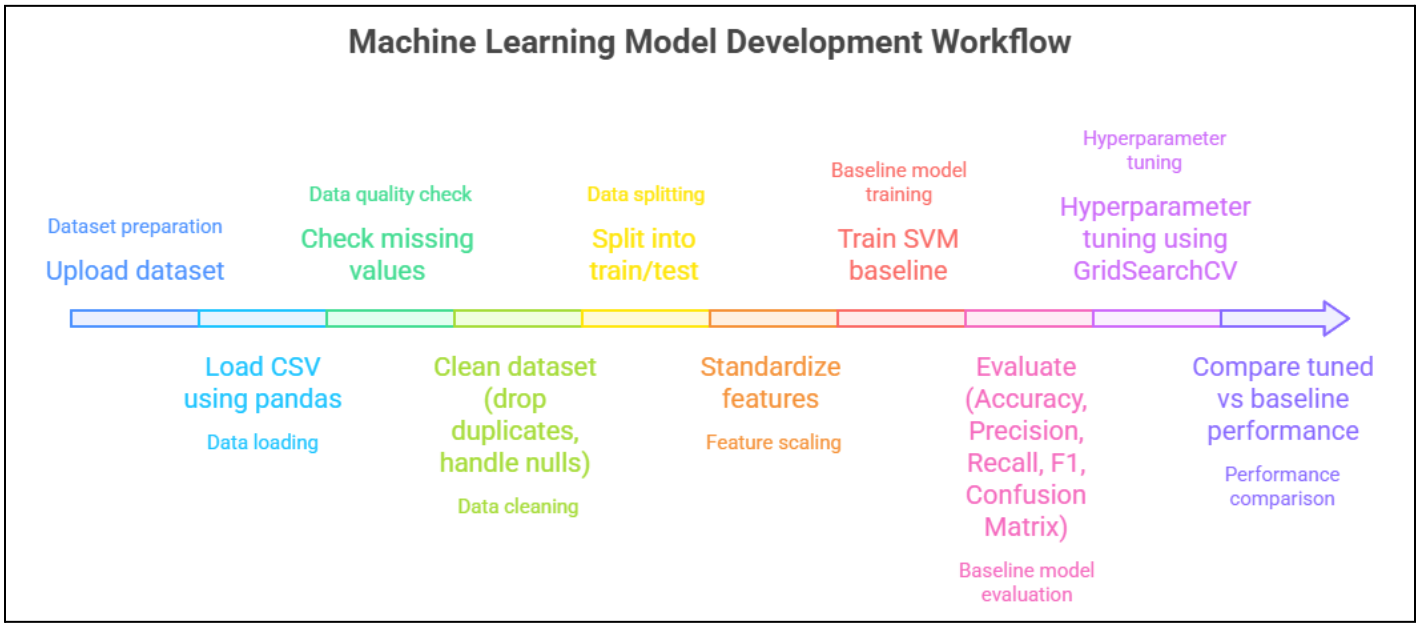
SVM becomes slow for **very large datasets**
Sensitive to **feature scaling** (must normalize/standardize)
Kernel selection is tricky
Hard to interpret compared to decision trees
Poor performance when classes overlap heavily

5. **Methodology / Workflow**

Steps:

1. Upload dataset
2. Load CSV using pandas
3. Check missing values
4. Clean dataset (drop duplicates, handle nulls)
5. Split into train/test
6. Standardize features
7. Train SVM baseline
8. Evaluate (Accuracy, Precision, Recall, F1, Confusion Matrix)
9. Hyperparameter tuning using GridSearchCV
10. Compare tuned vs baseline performance

Workflow Diagram (Text)



6. Performance Analysis

We will use:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report
-

7. Hyperparameter Tuning

We tune:

- C (regularization)
- kernel (linear / rbf / poly)
- gamma (kernel coefficient for rbf/poly)
- degree (for polynomial kernel)

We use:

GridSearchCV

Code and Output :

```
from google.colab import files
import pandas as pd
import io
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(io.BytesIO(uploaded[filename]))
print("Dataset Loaded Successfully!")
```

```
print("Shape:", df.shape)
df.head()
```

Choose Files heart.csv

heart.csv(text/csv) - 11328 bytes, last modified: 2/9/2026 - 100% done

```
Saving heart.csv to heart.csv
Dataset Loaded Successfully!
Shape: (303, 14)
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

```
print("Dataset Info:")
df.info()
print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Rows:", df.duplicated().sum())
# Remove duplicates
df.drop_duplicates(inplace=True)
print("\nShape after removing duplicates:", df.shape)
df.describe()
```

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trestbps  303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
 8   exang     303 non-null    int64
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64
 11  ca        303 non-null    int64
 12  thal      303 non-null    int64
 13  target    303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB

Missing Values:
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64

Duplicate Rows: 1

Shape after removing duplicates: (302, 14)
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
count	302.00000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000	302.000000
mean	54.42053	0.682119	0.963576	131.602649	246.500000	0.149007	0.526490	149.569536	0.327815	1.043046	1.397351	0.718543	2.314570	0.543046
std	9.04797	0.466426	1.032044	17.563394	51.753489	0.356686	0.526027	22.903527	0.470196	1.161452	0.616274	1.006748	0.613026	0.498970
min	29.00000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	48.00000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000	133.250000	0.000000	0.000000	1.000000	0.000000	2.000000	0.000000
50%	55.50000	1.000000	1.000000	130.000000	240.500000	0.000000	1.000000	152.500000	0.000000	0.800000	1.000000	0.000000	2.000000	1.000000
75%	61.00000	1.000000	2.000000	140.000000	274.750000	0.000000	1.000000	166.000000	1.000000	1.600000	2.000000	1.000000	3.000000	1.000000

```
X = df.drop("target", axis=1)
y = df["target"]
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (302, 13)
y shape: (302,)
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)
```

```
Training set: (241, 13)
Testing set: (61, 13)
```

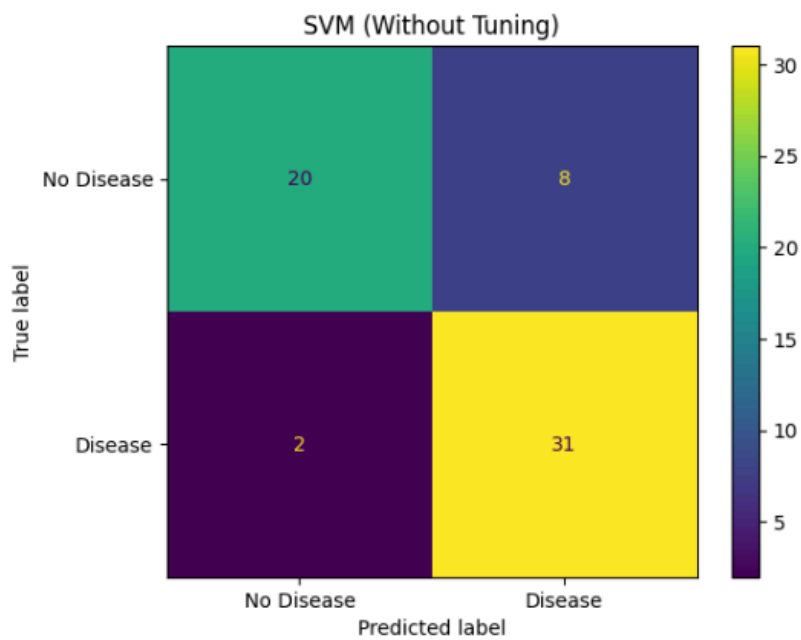
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("Scaling Done!")
```

```
Scaling Done!
```

```
# SVM WITHOUT Hyperparameter Tuning
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix,
ConfusionMatrixDisplay
import matplotlib.pyplot as plt
# Model
svm_model = SVC(kernel="rbf", C=1, gamma="scale", random_state=42)
# Train
svm_model.fit(X_train_scaled, y_train)
# Prediction
y_pred = svm_model.predict(X_test_scaled)
# Evaluation
print("🔹 SVM WITHOUT TUNING")
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred))
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["No Disease", "Disease"])
disp.plot()
plt.title("SVM (Without Tuning)")
plt.show()
```

```
◆ SVM WITHOUT TUNING
Accuracy: 0.8360655737704918

Classification Report:
               precision    recall  f1-score   support

           0       0.91      0.71      0.80        28
           1       0.79      0.94      0.86        33

    accuracy                           0.84        61
   macro avg       0.85      0.83      0.83        61
weighted avg       0.85      0.84      0.83        61
```



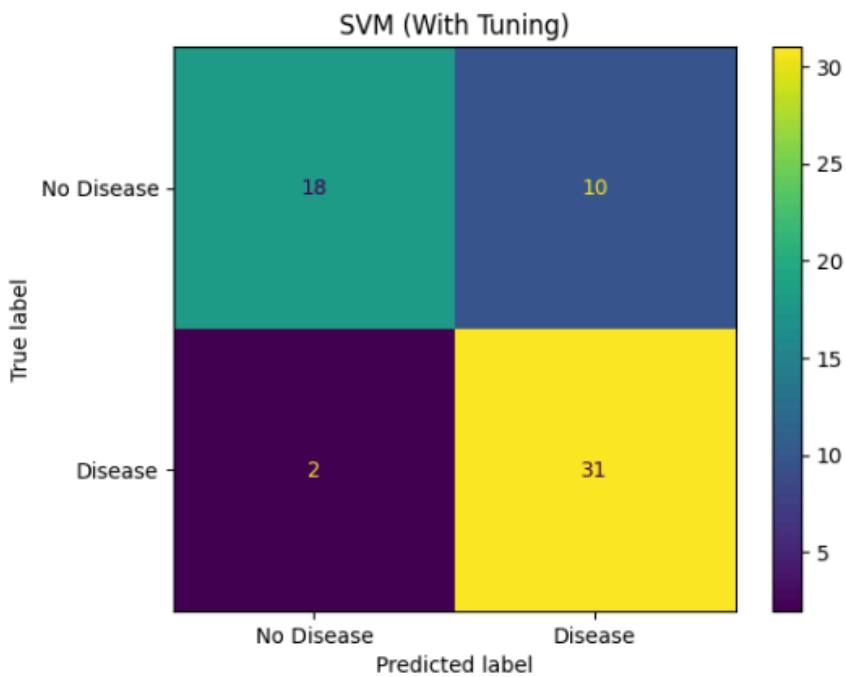

```
# SVM WITH Hyperparameter Tuning
from sklearn.model_selection import GridSearchCV
# Parameter Grid
param_grid = {
    "C": [0.1, 1, 10, 100],
    "kernel": ["linear", "rbf", "poly"],
    "gamma": ["scale", 0.01, 0.1, 1],
    "degree": [2, 3, 4]
}
```

```
# Grid Search
grid = GridSearchCV(
    SVC(),
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid.fit(X_train_scaled, y_train)

print("Best Parameters:", grid.best_params_)
print("Best CV Accuracy:", grid.best_score_)
```

```
Best Parameters: {'C': 1, 'degree': 2, 'gamma': 'scale', 'kernel': 'linear'}
Best CV Accuracy: 0.8342687074829932
```

```
# Tuned SVM Evaluation

best_svm = grid.best_estimator_

# Prediction
y_pred_tuned = best_svm.predict(X_test_scaled)

# Evaluation
print(" 🔹 SVM WITH TUNING")
print("Accuracy:", accuracy_score(y_test, y_pred_tuned))
print("\nClassification Report:\n", classification_report(y_test, y_pred_tuned))

# Confusion Matrix
cm_tuned = confusion_matrix(y_test, y_pred_tuned)
disp_tuned = ConfusionMatrixDisplay(confusion_matrix=cm_tuned, display_labels=["No Disease",
"Disease"])
disp_tuned.plot()
plt.title("SVM (With Tuning)")
plt.show()
```

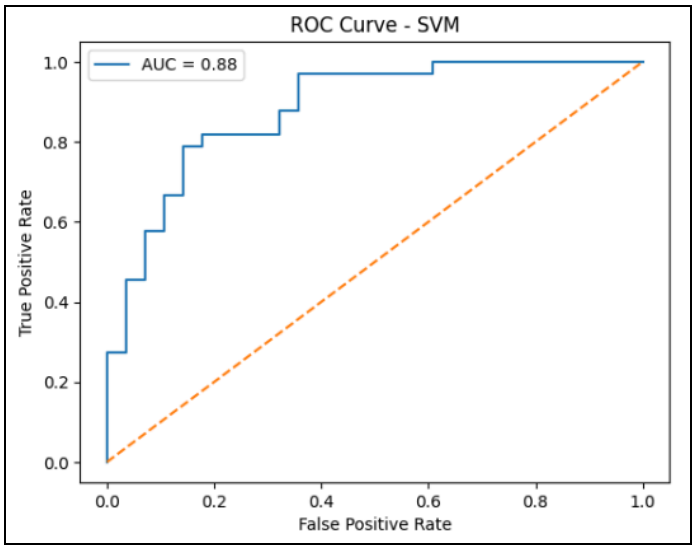
```
◆ SVM WITH TUNING
Accuracy: 0.8032786885245902

Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.64      0.75        28
           1       0.76      0.94      0.84        33

    accuracy                           0.80        61
   macro avg       0.83      0.79      0.79        61
weighted avg       0.82      0.80      0.80        61
```

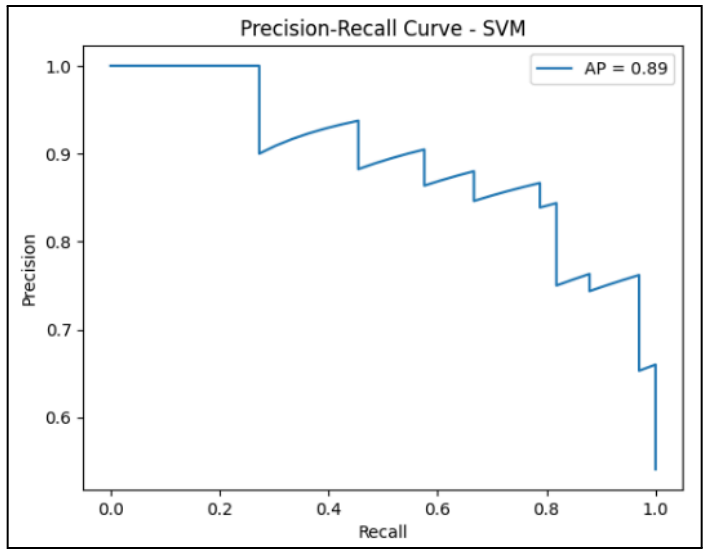


Performance Comparison

```
baseline_acc = accuracy_score(y_test, y_pred)
tuned_acc = accuracy_score(y_test, y_pred_tuned)

print("Baseline Accuracy:", baseline_acc)
print("Tuned Accuracy:", tuned_acc)
print("Improvement:", tuned_acc - baseline_acc)
```

```
Baseline Accuracy: 0.8360655737704918
Tuned Accuracy: 0.8032786885245902
Improvement: -0.032786885245901676
```





Google Colab Link for Code and Output : [Link for Code and Output]

Conclusion:

SVM performed well on the Heart Disease dataset because it is a medium-sized dataset with clear separable patterns. After applying feature scaling and tuning hyperparameters using GridSearchCV, the model achieved improved accuracy compared to the baseline model. The tuning process helped in selecting the best kernel and optimal values of C and gamma, which enhanced generalization performance.